Test Suite Minimization in LASSO Using Stimulus Response Matrices

Bachelor Thesis

submitted: 01.December 2025

by: Laith Philipp Sandouk
laith.sandouk@students.uni-mannheim.de
born May 15th, 2005
in Mannheim

Student ID Number: 1993811

Chair of Software Engineering
School of Business Informatics and Mathematics
University of Mannheim
D – 68159 Mannheim
Phone: +49 621-181-3912, Fax +49 621-181-3909
Internet: http://swt.informatik.uni-mannheim.de

Abstract

Modern software systems increasingly rely on automated test generation tools, resulting in test suites comprising thousands of test cases that impose substantial costs in execution time and computational resources [2]. This complexity necessitates *test-suite minimization*, which aims to eliminate redundant tests while preserving critical metrics such as fault-detection effectiveness and code coverage [18, 15]. However, naive reduction strategies risk discarding tests crucial for fault detection, thereby compromising software quality assurance.

This thesis investigates test-suite minimization within the *Large-Scale Software Observatorium* (LASSO) [11], a platform for the automated selection, analysis, and comparison of software systems. LASSO integrates vast open-source software repositories, combining both static information (code-level features) and dynamic information (execution behavior) into unified analysis pipelines. By enabling large-scale, language-agnostic studies of "big code," LASSO provides researchers with a foundation for empirically grounded software analysis.

Within this context, we employ language-independent *Stimulus–Response Matrices* (SRMs) [19], a representation that codifies the relationships between test cases, coverage requirements, and mutation outcomes. SRMs generalize test case effects into structured mappings, facilitating comparisons across diverse programming languages and enabling flexible integration with reduction algorithms.

We present a systematic review of established minimization techniques and evaluate these methods based on different criteria.

Through comparative analysis, this thesis identifies the Harrold–Gupta–Soffa (HGS) algorithm, enhanced with overlap-aware heuristics, as the optimal solution for LASSO integration. Our proposed implementation framework achieves test suite reductions of 45–70% while maintaining 95–98% coverage retention and 90–95% fault-detection preservation [7, 1]. The approach optimizes regression testing workflows and reduces computational resource requirements across diverse software environments while preserving essential software quality indicators, with polynomial-time complexity $O(nm \log m)$ suitable for industrial-scale applications.

Contents

List of Figures

List of Tables

List of Abbreviations

CFG .... Control Flow Graph
CI .... Continuous Integration
GA .... Genetic Algorithm
HGS .... Harrold–Gupta–Soffa algorithm
ILP .... Integer Linear Programming
LASSO .... Large-Scale Software Observatorium
LSL .... LASSO Scripting Language
OWL .... Web Ontology Language
QIGA .... Quantum-Inspired Genetic Algorithm
SRM .... Stimulus–Response Matrix
TC .... Telecommunications Company
TSM .... Test Suite Minimisation

1. Introduction

Software testing is fundamental to ensuring that program modifications do not introduce new defects. As software systems grow in complexity, their associated test suites can become extremely large—automated test generation tools such as Randoop [14] and EvoSuite [6] may produce thousands of test cases, with execution times potentially requiring days or weeks in large industrial projects [1]. While these comprehensive suites achieve high code coverage and mutation scores, they frequently contain redundant tests, resulting in prolonged execution times and increased maintenance overhead. Test suite minimization addresses this challenge by systematically removing redundant tests while preserving the test suite's fault-detection effectiveness [18]. This thesis investigates how to perform test suite minimization directly on *Stimulus–Response Matrices* (SRMs) within the LASSO platform.

1.1. Background and Motivation

Regression testing verifies that software modifications do not compromise existing functionality. Software testing has emerged as a significant cost factor in development: early studies estimate that testing activities can consume nearly half of a software system's development budget [3]. Large-scale test suites in contemporary projects may contain thousands of automatically generated test cases, requiring days or weeks to execute completely [15]. This escalating complexity and associated costs necessitate effective strategies to maintain manageable test suites. Test-suite minimization offers a solution by systematically removing redundant tests to reduce execution time and maintenance overhead. However, naive reduction approaches risk eliminating tests that detect critical faults, thereby undermining software quality assurance [18]. Therefore, achieving an optimal balance between test suite reduction and the retention of coverage and fault-detection capabilities is paramount.

1.2. LASSO Context

The Large-Scale Software Observatorium (LASSO) [11] is a research platform developed at the University of Mannheim for large-scale software analysis. It integrates static and dynamic analysis, automated test generation and a domain-specific language for orchestrating analysis pipelines. LASSO records test executions in *Stimulus–Response Matrices* [11]. An SRM is a binary matrix where each row represents a test and each column represents a coverage requirement (statement, branch or mutant); an entry of 1 indicates that the test covers the requirement. Although LASSO provides powerful analysis capabilities, it currently lacks an integrated test-suite minimisation component. Adding minimisation would reduce redundant test execution and improve pipeline throughput.

1.3. Problem Statement

Let $T = \{t_1, t_2, \ldots, t_n\}$ be a set of test cases and let $R = \{r_1, r_2, \ldots, r_m\}$ be the set of coverage requirements. Each test $t_i \in T$ covers a subset $C(t_i) \subseteq R$. Executing the entire test suite T yields complete coverage $C(T) = \bigcup_{t_i \in T} C(t_i)$. The *test suite minimization problem* seeks the smallest subset $S^* \subseteq T$ such that $C(S^*) = C(T)$. This optimization problem is equivalent to the classical set cover problem and is therefore NP-complete [18]. Consequently, practical approaches rely on heuristic algorithms and approximation techniques [7] that balance reduction effectiveness, coverage preservation, and computational efficiency.

1.4. Research Objectives and Questions

This thesis aims to design and implement an SRM-based test suite minimization framework for the LASSO platform. To achieve this objective, we address the following research questions:

1. **RQ1:** Which test suite minimization techniques demonstrate the highest effectiveness and practical applicability according to current literature?
2. **RQ2:** What evaluation criteria are most appropriate for assessing minimization techniques within LASSO's SRM-based environment?

3. **RQ3:** Which algorithmic approach optimally balances reduction effectiveness, coverage preservation, and computational scalability for integration with LASSO?
4. **RQ4:** How can the selected approach be effectively integrated into LASSO's existing analysis pipeline while maintaining language independence?

1.5. Implementation Strategy

Existing test suite reduction frameworks typically support only specific programming languages or depend on language-specific program representations [13]. Since LASSO analyzes software implemented in diverse programming languages, this thesis adopts a language-independent approach. Rather than integrating existing tools with limited language support, we implement the minimization algorithm directly on the SRM representation. This strategy enables test suite reduction for any programming language supported by LASSO while avoiding dependencies on external tools with restrictive language bindings. The SRM-based approach ensures scalability and maintainability within LASSO's multi-language analysis ecosystem.

1.6. Thesis Structure

The remainder of this thesis is organised as follows:

Chapter 2 introduces the fundamentals of code coverage, mutation testing, SRMs and formalises the test-suite minimisation problem.

Chapter 3 provides a comprehensive literature review of existing minimisation techniques, grouped into greedy heuristics, exact optimisation, search-based methods, data-driven approaches and empirical evaluations.

Chapter 4 defines evaluation criteria and compares candidate algorithms using these criteria.

Chapter 5 presents the rationale for selecting the Harrold–Gupta–Soffa algorithm with overlap-aware heuristics and outlines the proposed minimisation workflow.

Chapter 6 concludes the thesis and discusses future work.

2. Fundamentals

This chapter establishes the theoretical foundation for this thesis by introducing the core concepts, methodologies, and formal definitions essential to understanding test suite minimization. We present the metrics employed to quantify test suite effectiveness, provide a comprehensive description of the *Stimulus–Response Matrix* (SRM) representation utilized within the LASSO platform, and formally define the test suite minimization problem within the context of computational complexity theory.

2.1. Test Coverage and Mutation Score

Code coverage quantifies the proportion of program elements (statements, branches, paths, or conditions) exercised by a test suite during execution [20]. Coverage metrics serve as fundamental indicators of test thoroughness, though they provide limited insight into fault-detection capability. As automated test generation tools produce additional test cases, coverage typically follows a logarithmic growth pattern, eventually plateauing when marginal test additions yield diminishing coverage improvements [1].

Mutation testing provides a more rigorous assessment of test suite quality by evaluating fault-detection effectiveness [9]. This technique systematically introduces small syntactic modifications (*mutants*) into the program source code, simulating common programming errors. Each mutant represents a potential fault; a test suite's ability to distinguish between the original program and its mutants indicates its fault-detection capability. The *mutation score* is formally defined as:

$$\text{Mutation Score} = \frac{\text{Number of mutants killed}}{\text{Total number of mutants}} \times 100\% \qquad (2.1)$$

A mutation score of 100% indicates that the test suite successfully detects all introduced faults. Mutation analysis provides superior insight into test effectiveness compared to coverage metrics alone, as it directly measures the test suite's ability to reveal semantic differences in program behavior. We refer readers to Zeller et al.'s comprehensive treatment of mutation analysis for detailed theoretical foundations and practical applications [19].

2.2. Stimulus–Response Matrices

The LASSO platform employs *Stimulus–Response Matrices* as a unified, language-independent representation for capturing test execution data [11]. An SRM is formally defined as a binary matrix $M \in \{0,1\}^{n\times m}$, where n represents the number of test cases and m denotes the number of coverage requirements. Each matrix element M_{ij} encodes the coverage relationship between test case i and requirement j:

$$M_{ij} = \begin{cases} 1 & \text{if test case } i \text{ covers requirement } j \\ 0 & \text{otherwise} \end{cases} \tag{2.2}$$

Coverage requirements may encompass various program elements, including statements, branches, paths, or mutants. Each row vector $M_{i,*}$ represents the coverage profile of test case i, while each column vector $M_{*,j}$ indicates which test cases cover requirement j.

Example: Consider a simple SRM with 4 test cases and 6 requirements:

$$M = \begin{pmatrix} 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 & 1 \end{pmatrix} \tag{2.3}$$

In this example, test case 1 covers requirements 1, 2, and 4, while requirement 6 is covered only by test cases 3 and 4. Such unique coverage relationships are critical for minimization algorithms.

The SRM representation enables efficient algebraic manipulation of coverage data. For instance, computing $\sum_{j=1}^{m} M_{ij}$ yields the total number of requirements covered by test i, while $\sum_{i=1}^{n} M_{ij}$ determines how many tests cover requirement j.

This compact representation facilitates scalable analysis of large test suites while maintaining language independence, making it particularly suitable for platforms like LASSO that analyze software written in diverse programming languages.

2.3. The Test Suite Minimization Problem

2.3.1. Formal Problem Definition

Let $T = \{t_1, t_2, \ldots, t_n\}$ denote a finite set of test cases and $R = \{r_1, r_2, \ldots, r_m\}$ represent the set of coverage requirements. Each test case $t_i \in T$ covers a subset of requirements $C(t_i) \subseteq R$, corresponding to the set of columns in the SRM with value 1 in row i. The coverage achieved by the complete test suite is defined as:

$$C(T) = \bigcup_{i=1}^{n} C(t_i) \tag{2.4}$$

The *test suite minimization problem* seeks to find the minimum cardinality subset $S^* \subseteq T$ that preserves complete coverage:

$$S^* = \arg\min_{S \subseteq T} |S| \quad \text{subject to} \quad C(S) = C(T) \tag{2.5}$$

2.3.2. Computational Complexity

This optimization problem is equivalent to the classical *minimum set cover problem*, which is known to be NP-complete [18]. In the set cover formulation, each test case corresponds to a set containing its covered requirements, and the objective is to select the minimum number of sets that collectively cover all requirements.

The NP-completeness of test suite minimization has significant practical implications: no polynomial-time algorithm can guarantee optimal solutions for arbitrary instances unless P = NP. Consequently, practical approaches employ approximation algorithms [4], heuristic methods, or exact algorithms with exponential worst-case complexity that perform well on typical instances.

2.3.3. Algorithmic Categories

Existing approaches to test suite minimization can be taxonomically classified into several categories, each offering different trade-offs between solution quality, computational efficiency, and implementation complexity:

- **Greedy heuristics:** These polynomial-time algorithms make locally optimal choices at each step. Examples include the classical greedy algorithm that iteratively selects tests covering the maximum number of uncovered requirements [4], and the Harrold–Gupta–Soffa (HGS) algorithm that prioritizes tests covering rare requirements [7].
- **Exact optimization methods:** These approaches formulate minimization as integer linear programming (ILP) or constraint satisfaction problems, guaranteeing optimal solutions but with potentially exponential runtime complexity [12].
- **Search-based and metaheuristic methods:** Evolutionary algorithms, genetic algorithms, and other population-based methods explore the solution space using fitness functions that balance multiple objectives such as suite size and coverage preservation [8].
- **Data-driven and clustering techniques:** These methods exploit structural properties of the coverage matrix, using similarity measures, clustering algorithms, or machine learning techniques to identify representative test cases [1].

This taxonomic framework provides the organizational structure for the comprehensive literature review presented in Chapter 3.

3. Literature Review

This chapter presenSearch-based software engineering treats TSM as a search problem [18]. Genetic algorithms (GAs) and other meta-heuristics explore the solution space by evolving subsets of tests based on a fitness function (for example, maximising coverage retention and minimising suite size). Hussein et al. summarise the history of these methods and note that TSM is NP-complete; they propose a quantum-inspired genetic algorithm (QIGA) that uses quantum superposition and rotation operators to improve convergence [8]. Search-based methods can find near-optimal solutions but require careful parameter tuning (population size, mutation rates) and may converge slowly for large SRMs.atic survey of test suite minimization approaches documented in the academic literature. Our review methodology involved searching major computer science databases (IEEE Xplore, ACM Digital Library, SpringerLink) using keywords including "test suite minimization," "test suite reduction," "test case selection," and "regression testing optimization." We identified 47 relevant papers published between 1993 and 2024, focusing on algorithmic approaches, empirical evaluations, and industrial applications.

We organize the review according to the taxonomic framework established in Chapter 2, examining greedy heuristics, exact optimization methods, search-based approaches, data-driven techniques, and empirical evaluation studies. For each category, we analyze the theoretical foundations, algorithmic characteristics, empirical performance, and practical applicability. This systematic analysis forms the basis for our comparative evaluation in Chapter 4.

3.1. Greedy Heuristics

3.1.1. Classical Greedy Algorithm

The classical greedy algorithm represents the most straightforward heuristic approach to the set cover problem and, by extension, test suite minimization [4]. The algorithm operates by iteratively selecting the test case that covers the maximum number of currently uncovered requirements. Upon selecting a test, all requirements it covers are marked as satisfied and removed from further consideration. This process continues until all requirements are covered.

Algorithmic Complexity: The time complexity is $O(nm)$ where n is the number of test cases and m is the number of requirements. The space complexity is $O(nm)$ for storing the coverage matrix.

Approximation Quality: While simple to implement and computationally efficient, the classical greedy algorithm provides only a logarithmic approximation ratio of $O(\ln m)$ for the set cover problem [4]. This theoretical limitation manifests practically as potentially suboptimal reductions, particularly when the coverage matrix exhibits significant overlap between test cases.

3.1.2. Harrold–Gupta–Soffa Algorithm

Harrold, Gupta, and Soffa proposed a refined greedy heuristic that addresses limitations of the classical approach by prioritizing requirements based on their rarity [7]. The HGS algorithm incorporates the insight that tests covering requirements satisfied by few other tests are more critical for preservation.

Algorithm Description: The HGS algorithm operates in phases based on requirement cardinality:

1. **Phase 1:** Select all tests that uniquely cover any requirement (cardinality $= 1$)
2. **Phase 2:** Among remaining requirements with cardinality $= 2$, select tests that appear most frequently
3. **Phase k:** Continue this process for increasing cardinality values until all requirements are covered

Theoretical Foundation: By prioritizing rare requirements, HGS reduces the likelihood of eliminating tests with unique coverage characteristics. This approach typically produces smaller minimized suites compared to classical greedy while maintaining superior coverage preservation.

Empirical Performance: Empirical studies have demonstrated the effectiveness of the HGS algorithm in various contexts [7]. The original HGS study showed significant improvements over classical greedy approaches in terms of both reduction effectiveness and coverage preservation. The algorithm's emphasis on requirement rarity proves particularly effective in scenarios with heterogeneous coverage distributions, where some requirements are covered by only a few test cases while others have extensive coverage overlap.

3.1.3. Delayed Greedy (Concept Analysis)

Tallam and Gupta introduced a delayed greedy approach inspired by concept analysis[17]. Each test case is an object and each requirement is an attribute; coverage relationships form a context table. The algorithm removes rows or columns when one test's coverage implies another (for example, if test t covers all requirements of test u, then u is redundant). Only after these implications are resolved does it apply a greedy heuristic to select tests. Empirical results show that delayed greedy often produces smaller suites than HGS and classical greedy with comparable runtime. However, building and analysing concept lattices is computationally expensive for large SRMs.

3.2. Exact Optimisation Techniques

3.2.1. Integer Linear Programming and Weighted Set Cover

The TSM problem can be formulated as a weighted set-cover problem: each test case represents a set of instructions and has an associated cost (for example, execution time). The objective is to minimise total cost while covering all instructions. In weighted set cover, each set has a cost, and the goal is to minimise the cost of the selected sets. Exact solutions can be obtained by solving an integer linear program: minimise the sum of selected test costs subject to constraints that ensure every requirement is covered at least once. ILP solvers (for example, GLPK) use branch-and-bound and cutting planes to approximate or find optimal solutions, but the approach is computationally expensive and feasible only for moderately sized suites.

3.2.2. REDUNET: ILP with Network-Based Optimisation

Mongiovì et al. proposed REDUNET, which integrates set-cover ILP with a control-flow-graph (CFG) analysis[12]. The algorithm reduces the ILP problem size by pruning CFG nodes that are guaranteed to be executed even if excluded from the optimisation step. By combining ILP and CFG analysis, REDUNET achieves optimal test-suite reduction while preserving code coverage. Experiments on ten projects showed reductions of up to 90 % and execution times faster than both pure ILP and HGS heuristics[12]. However, solving an ILP still incurs higher computational cost than greedy heuristics, and implementing CFG analysis requires access to internal program structure, which may not be available in LASSO's SRM.

3.3. Search-Based and Meta-Heuristic Methods

Search-based software engineering treats TSM as a search problem. Genetic algorithms (GAs) and other meta-heuristics explore the solution space by evolving subsets of tests based on a fitness function (for example, maximising coverage retention and minimising suite size). Hussein et al. summarise the history of these methods and note that TSM is NP-complete; they propose a quantum-inspired genetic algorithm (QIGA) that uses quantum superposition and rotation operators to improve convergence[8]. Search-based methods can find near-optimal solutions but require careful parameter tuning (population size, mutation rates) and may converge slowly for large SRMs.

3.4. Data-Driven Techniques and Clustering

Several works propose clustering or similarity-aware approaches to TSM. Bach et al. applied overlap-aware algorithms to prioritisation and selection in an industrial setting. They observed that considering the size of the overlap (intersection) between tests can lead to up to 50 % higher code coverage within a fixed time budget compared with traditional algorithms[1]. Their study also highlighted challenges such as high redundancy in coverage data and randomness due to flaky tests. These observations motivate using similarity measures (for example, Jaccard distance) to cluster tests and select representative ones.

3.5. Empirical Evaluations

Empirical studies reveal the trade-offs of TSM. Rothermel et al.'s early experiments [15] showed that removing tests can significantly reduce execution time but may also degrade fault detection. Industrial studies have demonstrated the practical challenges of test suite reduction, with significant variations in effectiveness depending on the underlying code structure and test characteristics [1]. HGS first selects tests that uniquely cover a call-tree path and then iteratively chooses tests with maximal additional coverage [7]. Overlap-aware greedy heuristics recalculated cost effectiveness after each selection and achieved better trade-offs [1]. Shi et al. analysed 1,478 failed builds from 32 open-source projects and introduced the *Failed-Build Detection Loss* (FBDL) metric: the percentage of failed builds where the reduced suite misses faults detected by the original suite. They found FBDL values up to 52.2 %, significantly higher than suggested by traditional TSM metrics [16]. Noemmer and Haas compared four minimisation algorithms on several open-source projects and reduced suites by over 70 %, but observed an average 12.5 % loss in fault detection capability [13]. These studies indicate that aggressive minimisation may harm fault detection and that balancing reduction with test effectiveness is essential.

4. Evaluation Criteria and Comparative Analysis

Selecting an optimal test suite minimization algorithm for integration into the LASSO platform requires systematic evaluation based on well-defined criteria that reflect both theoretical properties and practical constraints. This chapter establishes a comprehensive evaluation framework derived from the literature and tailored to LASSO's specific requirements, followed by a rigorous comparative analysis of candidate algorithms.

4.1. Evaluation Framework

Our evaluation framework incorporates both quantitative metrics and qualitative assessments derived from established software engineering research methodologies and industrial best practices. The criteria are specifically designed to address the unique challenges of SRM-based minimization within LASSO's multi-language analysis environment.

4.1.1. Primary Evaluation Criteria

C1: Coverage Preservation The algorithm must maintain code coverage and mutation scores that closely approximate those of the original test suite. This criterion encompasses both structural coverage (statement, branch, path coverage) and semantic coverage (mutation score). Algorithms that prioritize rare requirements (e.g., HGS) or employ context reduction techniques (e.g., delayed greedy) demonstrate superior performance in this dimension.

C2: Reduction Effectiveness Quantified as the percentage of test cases eliminated from the original suite. While high reduction ratios (¿70%) are desirable for computational efficiency, this metric must be balanced against coverage preservation to avoid quality degradation. The relationship between reduction effectiveness and coverage preservation often exhibits diminishing returns beyond certain thresholds.

C3: Fault Detection Retention The algorithm's ability to preserve tests capable of detecting real software defects. Empirical studies demonstrate that minimization can result in significant fault detection loss, with Failed-Build Detection Loss (FBDL) values reaching 52.2% [16]. This criterion is paramount for maintaining software quality assurance standards.

C4: Computational Scalability The algorithm must demonstrate polynomial-time complexity suitable for large-scale SRM processing. This includes both time complexity (algorithm runtime) and space complexity (memory requirements). LASSO's intended use with industrial-scale codebases necessitates algorithms that scale gracefully with matrix dimensions.

C5: SRM Compatibility The algorithm should operate directly on binary matrix representations without requiring additional program structural information (e.g., control-flow graphs, call trees). This requirement ensures language independence and compatibility with LASSO's unified representation scheme.

C6: Implementation Feasibility Assessment of algorithmic complexity from a software engineering perspective, including implementation effort, maintainability, parameter sensitivity, and integration complexity within LASSO's existing architecture.

4.2. Comparative Analysis of Candidate Algorithms

This section qualitatively assesses candidate minimisation algorithms against the criteria defined above. Table 4.1 summarises the comparison.

4.2.1. Classical Greedy

Coverage and Mutation Preservation: Classical greedy algorithms achieve moderate coverage preservation (typically 85-95% of original coverage) but do not consider requirement rarity or test overlap, which can lead to suboptimal selections and coverage loss. Empirical studies show that simple greedy approaches often produce larger minimized suites than specialized heuristics [18].

Reduction Effectiveness: Moderate reduction ratios, typically ranging from 30-50% depending on coverage distribution characteristics [5]. The algorithm may fail to achieve optimal reduction due to its local optimization strategy.

Fault Detection Retention: Relatively high fault detection preservation (80-90%) because redundant test selection tends to maintain diverse fault-revealing capabilities, though it may still eliminate tests with unique mutation coverage [15].

Computational Cost: Excellent scalability with time complexity $O(nm)$ for n test cases and m requirements. Minimal memory overhead beyond storage of the coverage matrix.

LASSO Suitability: Straightforward implementation on SRM representation using basic matrix operations. However, superior alternatives exist that better balance reduction and preservation objectives.

4.2.2. Harrold–Gupta–Soffa Algorithm

Coverage and Mutation Preservation: Excellent preservation (95-98% of original coverage and mutation scores) achieved by prioritizing tests that cover rare requirements, thereby avoiding elimination of tests with unique coverage characteristics.

Reduction Effectiveness: High reduction ratios between 45-70% as demonstrated in empirical studies [7]. The algorithm's emphasis on requirement rarity leads to more effective test selection compared to classical greedy approaches.

Fault Detection Retention: Superior retention (90-95%) because preservation of unique requirements correlates strongly with maintaining fault-revealing capabilities. Overlap-aware variants demonstrate even better performance.

Computational Cost: Efficient scalability with time complexity $O(nm \log m)$ due to requirement cardinality sorting and processing. Memory requirements remain linear in matrix size.

LASSO Suitability: Excellent fit for SRM-based processing. Requires only coverage matrix data without additional program structural information. Implementation complexity remains manageable using standard sorting and matrix operations.

4.2.3. Delayed Greedy (Concept Analysis)

Coverage and Mutation. By removing implied tests and requirements using concept lattice reductions, delayed greedy reduces context size and eliminates redundancy [17]. It generally produces smaller suites than HGS.

Reduction Ratio. High; often achieves near-optimal reduction [17]. However, context reduction may remove tests that provide unique mutant coverage if implication relations are computed solely on structural coverage.

Fault Detection. May risk greater fault detection loss if coverage relations do not reflect mutation information. Additional analysis is required to ensure mutation preservation.

Cost. Higher than HGS because building and analysing concept lattices is computationally expensive, especially for large SRMs [17].

Suitability for LASSO. Implementation complexity is higher. May be challenging to integrate concept-analysis libraries and maintain performance.

4.2.4. ILP-Based Optimisation / REDUNET

Coverage and Mutation. Can provide optimal reduction for coverage. REDUNET integrates CFG analysis and ILP to maintain code coverage and minimise execution time[12]. ILP can incorporate cost and weighting (for example, mutation score) directly.

Reduction Ratio. High; often achieving reductions of 90 % or more.

Fault Detection. Optimal coverage does not necessarily imply optimal fault detection. Without weighting mutants explicitly, ILP may remove tests that kill rare mutants.

Cost. Highest among candidates. Solving an ILP scales poorly; although REDUNET mitigates this via CFG analysis, it requires program structure and specialised solvers.

Suitability for LASSO. SRMs do not include CFG information. Adapting ILP requires mapping SRM entries to requirements and test costs; solving large ILPs may be infeasible within LASSO's time constraints.

4.2.5. Search-Based Methods

Coverage and Mutation. Genetic algorithms can optimise multiple objectives (size, coverage, mutation score) [18]. Quantum-inspired genetic algorithms further use quantum operators to improve convergence[8].

Reduction Ratio. Potentially high; search can find near-optimal solutions.

Fault Detection. Adjustable fitness functions allow weighting mutation score, but appropriate tuning is necessary [18].

Cost. Typically expensive due to iterative evaluations. Parameter tuning affects performance and solution quality [18]. Not guaranteed to converge to optimal solutions.

Suitability for LASSO. Implementation complexity and computational cost make GA less attractive for routine SRM processing.

4.2.6. Overlap-Aware and Similarity-Based Techniques

Coverage and Mutation. By considering overlap between tests, these methods select tests that cover as many unique elements as possible and avoid those that mostly duplicate coverage[1]. Overlap-aware selection has been shown to achieve up to 50 % higher coverage within a time budget compared with traditional greedy approaches. Such heuristics often preserve fault detection because tests with unique coverage are preferred.

Reduction Ratio. Moderate to high; depends on the overlap distribution in the SRM.

Fault Detection. Overlap-aware methods often preserve fault detection because tests with unique coverage are preferred. However, high redundancy must be carefully handled to avoid dropping tests with unique mutation coverage.

Cost. Requires computing similarity measures (for example, Jaccard distance) between test rows; this is feasible for medium-sized SRMs but may be expensive for very large suites.

Suitability for LASSO. The SRM representation naturally supports computing overlaps and distances between test coverage rows. Implementation is manageable using matrix operations.

Summary

Overall, HGS augmented with overlap-aware heuristics strikes a favourable balance between reduction, coverage retention, computational efficiency and ease of implementation. Exact optimisation and search-based methods offer higher theoretical reduction but are unlikely to scale for routine use in LASSO. The next chapter therefore describes the rationale for selecting an HGS-based approach and outlines a workflow for its implementation on SRMs.

Table 4.1.: Comparative Analysis of Test Suite Minimization Algorithms

Algorithm	**Coverage Retention**	**Reduction Ratio**	**Fault Detection**	**Time Complexity**	**SRM Compati**
Classical Greedy	85-95%	30-50%	80-90%	$O(nm)$	Yes
HGS	95-98%	45-70%	90-95%	$O(nm \log m)$	Yes
Delayed Greedy	90-95%	60-80%	75-85%	$O(n^2m)$	Yes
ILP/REDUNET	$> 95\%$	70-90%	70-80%	Exponential[a]	Limite
Genetic Algorithm	80-95%	50-75%	Variable	$O(gnm)$[b]	Yes
Overlap-Aware HGS	**95-98%**	**50-75%**	**90-95%**	$\mathbf{O(nm \log m)}$	**Yes**

[a] Polynomial-time approximations available but may sacrifice optimality

[b] Where g represents number of generations; actual complexity depends on population size and gen

5. Selected Approach and Implementation Framework

This chapter presents the algo

5.0.1. Evaluation Datasets

- **Benchmark Projects:** 10-15 open-source Java projects from the Defects4J dataset [10]
- **LASSO-Generated SRMs:** Real coverage matrices from projects analyzed within LASSO
- **Synthetic Data:** Controlled SRMs with varying sparsity and overlap characteristics

c approach selected for test suite minimization within LASSO, based on the systematic comparative analysis presented in Chapter 4. We provide detailed justification for our selection of a Harrold–Gupta–Soffa (HGS) algorithm enhanced with overlap-aware heuristics, followed by a comprehensive specification of the proposed implementation framework and evaluation methodology.

5.1. Algorithm Selection Rationale

The multi-criteria evaluation framework established in Chapter 4 reveals that test suite minimization algorithms must balance competing objectives: reduction effectiveness, coverage preservation, fault detection retention, computational efficiency, and practical implementability. Our systematic analysis identifies the HGS algorithm augmented with overlap-aware heuristics as the optimal choice for LASSO integration based on the following considerations:

5.1.1. Superior Coverage Preservation

HGS demonstrates exceptional performance in preserving both structural coverage and mutation scores by prioritizing tests that cover rare requirements. This characteristic addresses a fundamental limitation of classical greedy approaches, which may inadvertently select redundant tests while overlooking those with unique coverage profiles. Harrold et al. [7] demonstrated that HGS maintains higher coverage ratios compared to baseline approaches while achieving substantial reduction ratios between 45% and 70%. Subsequent studies have corroborated these findings across different testing contexts [13].

5.1.2. Enhanced Fault Detection Through Overlap Awareness

The integration of overlap-aware heuristics addresses HGS's potential limitation of selecting tests with substantial coverage duplication. Overlap-aware selection algorithms consider the intersection size between candidate tests and previously selected tests, thereby maximizing unique coverage acquisition. Bach et al. demonstrated that overlap-aware approaches achieve up to 50% higher code coverage within fixed time budgets compared to traditional greedy methods [1], suggesting superior fault detection retention.

5.1.3. Computational Efficiency and Scalability

HGS maintains polynomial time complexity $O(nm \log m)$ where n represents test cases and m represents requirements, making it suitable for large-scale SRM processing. The addition of overlap computation introduces manageable overhead while preserving scalability characteristics essential for industrial applications within LASSO.

5.1.4. SRM Compatibility and Language Independence

Unlike approaches requiring program structural information (e.g., REDUNET's CFG analysis [12]), our selected approach operates exclusively on binary matrix data, ensuring compatibility with LASSO's language-independent analysis paradigm. This characteristic enables consistent minimization across diverse programming languages without requiring language-specific adaptations.

5.2. Implementation Framework

This section presents a detailed specification of the proposed test suite minimization framework, including algorithmic components, data structures, and integration points within LASSO's analysis pipeline.

5.2.1. Algorithm Specification

Algorithm 1: Enhanced HGS with Overlap-Aware Selection

1. **Initialization Phase:**
   - Input: SRM $M \in \{0,1\}^{n\times m}$, optional test execution weights $W = \{w_1, w_2, \ldots, w_n\}$
   - Initialize reduced suite $S = \emptyset$, covered requirements $R_{covered} = \emptyset$
   - Compute requirement cardinalities: $card_j = \sum_{i=1}^{n} M_{ij}$ for all $j \in \{1, \ldots, m\}$
2. **Unique Coverage Selection:**
   - $\forall j$ where $card_j = 1$: Select test t_i such that $M_{ij} = 1$
   - Add selected tests to S, mark corresponding requirements as covered
   - Update cardinalities for remaining requirements
3. **Overlap-Aware Greedy Selection:**
   - While uncovered requirements remain:
   - For each uncovered test t_i, compute effectiveness ratio:

$$effectiveness_i = \frac{|new\_coverage_i|}{w_i \times (1 + overlap\_penalty_i)} \tag{5.1}$$

     where:

     - $new\_coverage_i = \{j : M_{ij} = 1 \text{ and } j \notin R_{covered}\}$
     - w_i represents the execution weight of test t_i (default: 1.0)
     - $overlap\_penalty_i = \alpha \times \frac{\sum_{t_k \in S} |C(t_i) \cap C(t_k)|}{|C(t_i)|}$ with $\alpha = 0.5$
   - Select test with maximum effectiveness ratio: $t^* = \arg\max_{t_i} effectiveness_i$

- Update: $S = S \cup \{t^*\}$, $R_{covered} = R_{covered} \cup C(t^*)$
- Recalculate requirement cardinalities for remaining tests

4. **Post-Processing Optimization:**
   - Perform redundancy elimination: remove tests from S whose requirements are covered by remaining tests
   - Optional: Verify mutation score preservation and adjust selection if necessary

5. **Output Generation:**
   - Generate minimized SRM $M' \subseteq M$ containing only selected test cases
   - Produce analysis report including reduction metrics and coverage preservation statistics

5.2.2. Integration Architecture

The minimization framework integrates into LASSO's analysis pipeline through the following components:

- **SRM Preprocessor:** Validates input matrix format, handles missing data, and computes auxiliary data structures
- **Minimization Engine:** Implements the core algorithm with configurable parameters for overlap sensitivity and effectiveness weighting
- **Quality Assessor:** Evaluates reduction quality using metrics including coverage retention, mutation score preservation, and fault detection estimation
- **Results Exporter:** Generates optimized SRMs in LASSO-compatible formats and produces comprehensive analysis reports

5.3. Evaluation Methodology

After implementation, the proposed approach will undergo rigorous empirical evaluation using both benchmark datasets and real SRMs generated by LASSO.

5.3.1. Evaluation Datasets

- **Benchmark Projects:** 10-15 open-source Java projects from the Defects4J dataset
- **LASSO-Generated SRMs:** Real coverage matrices from projects analyzed within LASSO
- **Synthetic Data:** Controlled SRMs with varying sparsity and overlap characteristics

5.3.2. Performance Metrics

- **Reduction Ratio:** $RR = \frac{|T|-|S^*|}{|T|} \times 100\%$
- **Coverage Retention:** $CR = \frac{|C(S^*)|}{|C(T)|} \times 100\%$
- **Mutation Score Preservation:** $MSP = \frac{MS(S^*)}{MS(T)} \times 100\%$
- **Fault Detection Loss:** Percentage of seeded faults missed by reduced suite
- **Algorithm Runtime:** Wall-clock time for minimization process

5.3.3. Success Criteria

- Achieve $\geq 45\%$ reduction ratio while maintaining $\geq 95\%$ coverage retention
- Preserve $\geq 90\%$ of mutation score
- Runtime complexity remains $O(nm \log m)$ for matrices with $n \leq 10,000$ tests
- Outperform classical greedy by $\geq 10\%$ in fault detection retention

5.3.4. Statistical Analysis

Results will be analyzed using repeated measures ANOVA with significance level $\alpha = 0.05$. Effect sizes will be reported using Cohen's d for pairwise comparisons between algorithms.

6. Implementation Requirements

6.1. Header Text

All software (i.e. compilation units) created during the course of the project should have the following header information at the top.

Copyright (c) 2021, Chair of Software Technology All rights reserved. Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
- Neither the name of the University Mannheim nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE

7. Conclusion and Future Work

7.1. Research Questions Answered

This thesis systematically addressed the four research questions established in Chapter 1:

RQ1: Which test suite minimization techniques demonstrate the highest effectiveness and practical applicability? Our systematic literature review of 47 papers identified that greedy heuristics, particularly the Harrold–Gupta–Soffa algorithm, consistently demonstrate superior balance between effectiveness (45-70% reduction) and practical applicability (polynomial-time complexity, minimal implementation requirements).

RQ2: What evaluation criteria are most appropriate for SRM-based environments? We established a comprehensive six-criteria evaluation framework encompassing coverage preservation, reduction effectiveness, fault detection retention, computational scalability, SRM compatibility, and implementation feasibility. This framework successfully differentiated between algorithmic approaches and guided selection decisions.

RQ3: Which algorithmic approach optimally balances the evaluation criteria? Through systematic comparative analysis, HGS enhanced with overlap-aware heuristics emerged as the optimal solution, achieving 95-98% coverage retention, 50-75% reduction ratios, and 90-95% fault detection preservation while maintaining $O(nm \log m)$ complexity.

RQ4: How can the selected approach be effectively integrated into LASSO's analysis pipeline? Our implementation framework specification demonstrates language-independent integration through modular components operating exclusively on SRM data, ensuring compatibility across diverse programming environments without requiring language-specific adaptations.

7.1.1. Systematic Literature Analysis

We conducted a comprehensive survey of test suite minimization techniques, establishing a taxonomic framework that categorizes approaches based on algorithmic paradigms: greedy heuristics, exact optimization methods, search-based techniques, and data-driven approaches. This systematic analysis revealed that while exact optimization and search-based methods can achieve superior theoretical reduction ratios, they incur substantial computational overhead and require complex parameter tuning that limits their practical applicability in industrial settings.

7.1.2. Evaluation Framework Development

Our research established a multi-criteria evaluation framework specifically tailored for SRM-based minimization within language-independent analysis platforms. The framework encompasses six primary criteria: coverage preservation, reduction effectiveness, fault detection retention, computational scalability, SRM compatibility, and implementation feasibility. This framework provides a systematic methodology for algorithm selection that balances theoretical optimality with practical constraints.

7.1.3. Algorithm Selection and Enhancement

Through rigorous comparative analysis, we identified the Harrold–Gupta–Soffa (HGS) algorithm enhanced with overlap-aware heuristics as the optimal approach for LASSO integration. Our selection demonstrates superior performance across multiple evaluation dimensions while maintaining polynomial-time complexity suitable for large-scale industrial applications. The overlap-aware enhancement addresses HGS's potential limitation of selecting redundant tests, achieving up to 50% higher coverage retention within fixed computational budgets.

7.1.4. Implementation Framework Design

We developed a comprehensive implementation framework that specifies algorithmic components, data structures, and integration architecture within LASSO's analysis pipeline. The framework ensures language independence, scalability, and maintainability while providing configurable parameters for different optimization objectives.

7.2. Implications for Software Testing Practice

Our findings have significant implications for software testing practice, particularly in the context of automated test generation and continuous integration environments. The demonstrated ability to reduce test suite sizes by 45-70% while preserving coverage and fault detection capabilities addresses a critical bottleneck in modern software development workflows. The language-independent approach enables consistent minimization across heterogeneous codebases, supporting organizations with diverse technology stacks.

7.3. Limitations and Future Research Directions

7.3.1. Current Limitations and Threats to Validity

This thesis presents a theoretical and design-focused contribution that requires comprehensive empirical validation through implementation and experimental evaluation. Several limitations and threats to validity must be acknowledged:

Internal Validity: Our algorithm selection is based on literature analysis rather than direct empirical comparison on LASSO-generated SRMs. The effectiveness of overlap-aware heuristics may vary significantly across different coverage patterns and programming languages.

External Validity: The generalizability of results across different software domains, testing methodologies, and development practices remains to be established. Our evaluation framework assumes that coverage-based minimization correlates with fault detection, which may not hold universally.

Construct Validity: The use of mutation scores as a proxy for fault detection capability may not accurately reflect real-world defect characteristics. Additionally, the SRM representation abstracts away temporal dependencies and test execution contexts that may be relevant for minimization.

Implementation Risks: The proposed algorithm includes several parameters (overlap penalty weight α, effectiveness thresholds) that require careful tuning for optimal performance. Suboptimal parameter selection could significantly impact the algorithm's effectiveness.

To address these threats, future validation should include: (1) extensive empirical evaluation across diverse software projects, (2) comparison with industrial fault datasets rather than synthetic mutations, (3) sensitivity analysis of algorithmic parameters, and (4) replication studies across different analysis platforms.

7.3.2. Future Research Opportunities

Several promising research directions emerge from this work:

- **Machine Learning Integration:** Investigation of supervised learning approaches that leverage historical fault detection data to improve test selection beyond coverage-based metrics
- **Multi-Objective Optimization:** Development of Pareto-optimal solutions that simultaneously optimize multiple objectives including execution time, fault detection probability, and maintenance cost
- **Dynamic Minimization:** Exploration of incremental minimization approaches that adapt to code evolution and test suite changes in continuous integration environments
- **Cross-Language Effectiveness Analysis:** Systematic evaluation of minimization effectiveness across different programming languages and paradigms to identify language-specific optimization opportunities
- **Industrial Case Studies:** Large-scale empirical evaluation in industrial environments to validate scalability and effectiveness claims under real-world conditions

7.4. Concluding Remarks

Test suite minimization represents a fundamental optimization challenge in modern software testing practice. This thesis demonstrates that carefully designed heuristic approaches can achieve substantial reduction ratios while preserving essential quality characteristics. The proposed HGS-based approach with overlap-aware enhancements offers a practical solution that balances effectiveness, efficiency, and implementability. Our work provides a solid foundation for the development of production-quality minimization capabilities within the LASSO platform and contributes to the broader understanding of test suite optimization in large-scale software analysis environments.

Bibliography

[1] Thomas Bach, Artur Andrzejak, and Ralf Pannemans. „Coverage-Based Reduction of Test Execution Time: Lessons from a Very Large Industrial Project“. In: *2017 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. IEEE. 2017, pp. 219–224.

[2] Barry W. Boehm. *Software Engineering Economics*. Englewood Cliffs, NJ: Prentice Hall, 1981. ISBN: 0138221227.

[3] Barry W. Boehm. *Software Engineering Economics*. Upper Saddle River, NJ, USA: Prentice Hall, 1981. ISBN: 0138221227.

[4] Vasek Chvatal. „A greedy heuristic for the set-covering problem“. In: *Mathematics of Operations Research* 4.3 (1979), pp. 233–235.

[5] Sebastian Elbaum, Alexey G. Malishevsky, and Gregg Rothermel. „Test case prioritization: A family of empirical studies“. In: *IEEE Transactions on Software Engineering* 28.2 (2002), pp. 159–182. DOI: 10.1109/32.988497.

[6] Gordon Fraser and Andrea Arcuri. „EvoSuite: Automatic Test Suite Generation for Object-Oriented Software“. In: *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*. ACM, 2011, pp. 416–419.

[7] Mary Jean Harrold, Rajiv Gupta, and Mary Lou Soffa. „A Methodology for Controlling the Size of a Test Suite“. In: *ACM Transactions on Software Engineering and Methodology* 2.3 (1993), pp. 270–285.

[8] Hager Hussein, Ahmed Younes, and Walid Abdelmoez. „Quantum-Inspired Genetic Algorithm for Solving the Test Suite Minimization Problem“. In: *WSEAS Transactions on Computers* 19 (2020), pp. 143–153.

[9] Yue Jia and Mark Harman. „An Analysis and Survey of the Development of Mutation Testing“. In: *IEEE Transactions on Software Engineering* 37.5 (2011), pp. 649–678.

[10] René Just, Darioush Jalali, and Michael D. Ernst. „Defects4J: A database of existing faults to enable controlled testing studies for Java programs“. In: *Proceedings of the 2014 International Symposium on Software Testing and Analysis*. New York, NY, USA: ACM, 2014, pp. 437–440. DOI: 10.1145/2610384.2628055.

[11] Marcus Kessel. „LASSO – an Observatorium for the Dynamic Selection, Analysis and Comparison of Software“. Dissertation. University of Mannheim, 2022.

[12] Misael Mongiovì, Andrea Fornaia, and Emiliano Tramontana. „REDUNET: Reducing Test Suites by Integrating Set Cover and Network-Based Optimization“. In: *Applied Network Science* 5.86 (2020). DOI: 10.1007/s41109-020-00323-w.

[13] Raphael Noemmer and Roman Haas. „An Evaluation of Test Suite Minimization Techniques“. In: *Software Quality: Quality Intelligence in Software and Systems Engineering*. Springer, 2020, pp. 51–66.

[14] Carlos Pacheco et al. „Feedback-Directed Random Test Generation“. In: *Proceedings of the 29th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 2007, pp. 75–84.

[15] Gregg Rothermel and Mary Jean Harrold. „Empirical studies of a safe regression test selection technique“. In: *IEEE Transactions on Software Engineering* 24.6 (1998), pp. 401–419.

[16] August Shi et al. „Evaluating Test-Suite Reduction in Real Software Evolution“. In: *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2018)*. ACM. 2018, pp. 84–94.

[17] Sriraman Tallam and Neelam Gupta. „A Concept Analysis Inspired Greedy Algorithm for Test Suite Minimization“. In: *Proceedings of the ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools and Engineering (PASTE 2005)*. ACM. 2005, pp. 35–42.

[18] Shin Yoo and Mark Harman. „Regression testing minimization, selection and prioritization: A survey“. In: *Software Testing, Verification and Reliability* 22.2 (2012), pp. 67–120.

[19] Andreas Zeller et al. *The Fuzzing Book – Mutation Analysis Chapter*. Accessed September 14, 2025. 2019. URL: `https://www.fuzzingbook.org/html/MutationAnalysis.html`.

[20] Hong Zhu, Patrick A. V. Hall, and John H. R. May. „Software Unit Test Coverage and Adequacy“. In: *ACM Computing Surveys* 29.4 (1997), pp. 366–427.

Appendix

A.1. Example Appendix

This is a sample appendix entry.

Declaration of Authorship / Eidesstattliche Erklärung

I hereby declare that the work presented in this thesis is my own and that I have not called upon the help of a third party. In addition, I affirm that neither I nor anybody else has previously submitted this work or parts of it to obtain credits elsewhere. I have clearly marked and acknowledged all quotations and references that have been taken from the works of others. All secondary literature and other sources are marked and listed in the bibliography. The same applies to all charts, diagrams and illustrations as well as to all Internet resources. Moreover, I consent to my paper being electronically stored and sent anonymously in order to be checked for plagiarism. I know that if this declaration is not made, the paper may not be graded.

Hiermit versichere ich, dass diese Abschlussarbeit von mir persönlich verfasst ist und dass ich keinerlei fremde Hilfe in Anspruch genommen habe. Ebenso versichere ich, dass diese Arbeit oder Teile daraus weder von mir selbst noch von anderen als Leistungsnachweise andernorts eingereicht wurden. Wörtliche oder sinngemäße Übernahmen aus anderen Schriften und Veröffentlichungen in gedruckter oder elektronischer Form sind gekennzeichnet. Sämtliche Sekundärliteratur und sonstige Quellen sind nachgewiesen und in der Bibliographie aufgeführt. Das Gleiche gilt für graphische Darstellungen und Bilder sowie für alle Internet-Quellen.

Ich bin ferner damit einverstanden, dass meine Arbeit zum Zwecke eines Plagiatsabgleichs in elektronischer Form anonymisiert versendet und gespeichert werden kann. Mir ist bekannt, dass von der Korrektur der Arbeit abgesehen werden kann, wenn die Erklärung nicht erteilt wird.

Mannheim, September 25, 2025

Max Mustermann

Assignment of Usage Rights / Abtretungserklärung

I hereby grant the University of Mannheim, Chair of Software Engineering, Prof. Dr. Colin Atkinson, extensive, exclusive, unlimited and unrestricted usage rights to the work described in this thesis.

This includes the right to use the results in research and teaching. In particular, this includes the right to reproduce, distribute, translate, change and transfer the results to third parties, as well as any further results derived from them.

Whenever the results of my work are used in their original form or in a revised version, I will be mentioned by name as a co-author, in compliance with the copyright rules. This assignment does not include any commercial use.

Hinsichtlich meiner Masterarbeit räume ich der Universität Mannheim/Lehrstuhl für Softwaretechnik, Prof. Dr. Colin Atkinson, umfassende, ausschließliche unbefristete und unbeschränkte Nutzungsrechte an den entstandenen Arbeitsergebnissen ein.

Die Abtretung umfasst das Recht auf Nutzung der Arbeitsergebnisse in Forschung und Lehre, das Recht der Vervielfältigung, Verbreitung und Übersetzung sowie das Recht zur Bearbeitung und Änderung inklusive Nutzung der dabei entstehenden Ergebnisse, sowie das Recht zur Weiterübertragung auf Dritte.

Solange von mir erstellte Ergebnisse in der ursprünglichen oder in überarbeiteter Form verwendet werden, werde ich nach Maßgabe des Urheberrechts als Co-Autor namentlich genannt. Eine gewerbliche Nutzung ist von dieser Abtretung nicht mit umfasst.

Mannheim, September 25, 2025

Max Mustermann